

[Collaborative Docs](#) >

Discussion Agenda 06/13/2014 (Grackle + PoC-NOSUBGRID)

Participants of Paper Group "3" discussed the items described below on June 13, 2014 (Fri, 9am PDT, 6pm CEST). Two major issues are:

- [1] Grackle implementation
- [2] Launching of Proof-of-Concept-NOSUBGRID runs ($10^{11} M_{\text{sun}}$ halo runs without subgrid physics)

Contents

- [1 1. Some Observations](#)
- [2 2. Participants Today](#)
- [3 3. Grackle Implementation - Proposed Solutions](#)
- [4 4. Useful Links](#)

Check out the latest actions in red color below!

(1st revision: 06/15/2014, 2nd revision: 06/20/2014, 3rd revision: 08/13/2014)

1. Some Observations

(1) It was promised in the Flagship paper that we would all use Grackle, and half the code groups (ART-II, ENZO, GADGET) already use Grackle successfully (in [Pure Tabulated Mode](#)). Remaining issues tend to be about technical aspects and interfaces.

(2) The Grackle issue became a major bottleneck for progress in Papers 3 and 4, and anything beyond. Let us keep constructive tone as we always have done, and determine action items to move forward with Paper 3.

2. Participants Today

(1) Paper "3" leaders: Gnedin, Smith, Madau

(2) Paper "3" code point persons:

Code	Point persons (black if joined the telecon)
ART-I	Ceverino, Klypin
ART-II	Leitner, Gnedin
CHANGA	Quinn
ENZO	Hummels, Kim, Simpson, Smith, Turk , Wise
GADGET	Choi, Hobbs , Nagamine, Thompson, Todoroki
GASOLINE	Madau, Mayer , Shen , Wadsley
RAMSES	Agertz, Teyssier

3. Grackle Implementation - Proposed Solutions

(1)

• Interface Issues (raised by Gnedin et al.)

- Some arguments required by Grackle are not relevant for certain codes, and may create overhead.

• **Proposal #1 (by Gnedin, Hummels, Smith et al.) --> All the code representatives agreed that this will likely work. If it doesn't work for some codes, we may then discuss Proposal #2 below.**

– [Step 1] First, each code representative decides on the appropriate API for Grackle which has sufficient arguments to do what they want, but not too many that may bog down their function calls. Ideally, we all converge on a single API, but if that is not possible, then there may be different API for some of the codes, for example `agorasolvechemistry(...)` as a common API, `agorasolvechemistry_enzo(...)` just for Enzo, etc. For example, `agorasolvechemistry()` passes cooling rates for individual resolution elements, while `agorasolvechemistry_enzo()` accepts patches. Once API is finalized, each code representative is expected to modify his/her code to accept this specific function call.

– [Step 2] Then, each code representative run a test of a $10^{11} M_{\text{sun}}$ halo down to $z=8$, with Gnedin's dummy API (link below) that only returns zeroes for cooling, in order to make sure that the run proceeds adiabatically.

– [Step 3] If all tests are successful, Gnedin has volunteered to write wrappers around the existing Grackle, which will map the existing `solve_chemistry()` function in Grackle to all of the specific ones expected by the API.

• **Action Item: Let us complete Step 1 and report the result to Gnedin by 07/31. Commit your change to Gnedin's dummy Grackle [repository](#) accordingly. When you do so, keep in mind the following.**

– [1] If you introduce a function call for your specific code, then you are, of course, free to choose the argument list as you see fit. But, please, only do that as a last resort, if you are unable to use the common API for a really good reason (e.g. need a field that is not available).

– [2] If you are modifying the common API, make sure that all calls are wrapped into `f77call(...)` macro, use only lower case letters and numbers for function names, and specify all arguments as pointers to float, double, or int ONLY, so that these functions can be called both from C and Fortran.

– [3] You can assume that the implementation of `agorasolvechemistry()` is thread-safe (i.e. can be called from an OpenMP loop).

– [4] The header file should conform to C99 standard; please, no C++ code there.

– [5] If your code currently takes into account an “external heating” term when the thermal energy equation is solved (e.g. pdV work, or in some cases, SNe energy), consider adding the pdV work separately from the cooling solver, and before it (see note 1). In other words, perform the cooling and pdV calculation in 2 steps:

1. $E_{1/2} = E_0 + \text{pdV}$
2. $E_1 = \text{agorasolvechemistry}(E_{1/2})$

However, if it is convenient for you to add pdV term in the cooling function call, by all means do that – just modify the API appropriately. Then we will think about the order of sub-steps in the implementation.

[note 1] If pdV work is included in a cooling solver, you might experience numerical problem of overcooling on shocks. For example, if the shock takes primordial gas from $T=10^3$ to 10^5 K, there should not be much cooling happening. However, if the pdV work is included in the cooling solver, then the gas passing through the artificially smoothed shock may only reach just above 10^4 K, and cooling may become very efficient. There is no complete solution to this problem, but, splitting pdV from the cooling solver mitigates the numerical error somewhat. (authored by Gnedin)

(2)

• Other Issues (raised by Teyssier et al.)

– As of now, one needs to define two structs in the code in addition to calling Grackle. One struct (`code_units`) stores the code-unit-to-CGS-unit information, and the other (`chemistry_data`) contains the actual lookup table of the cooling information created during `initialize_chemistry_data()`. There is a concern that creating these structs may not fit with certain Fortran codes. It would be ideal if Grackle has just two function calls (`solve_chemistry()`, `initialize_chemistry_data()`) with no need to predefine structs in their code than to include `grackle.h`. Some suggested these to be hidden from the user by making Grackle as an entirely externally-compiled library. (lookup table computed during compilation?)

– Write Grackle in C instead of C++ so it would be more easily called from Fortran codes?

– Different time integration scheme in each code, not compatible with a single common cooling routine?

– The reality is that some code groups do not have the manpower to implement Grackle in their codes. In order to put the entire AGORA Project on a path to a continued success these groups may need extra manpower from within the Collaboration to complete this task.

• **Ongoing Actions: Being carried out by Smith, see the Grackle [discussion](#) on 06/20/2014, and the [announcement](#) of new API on 08/13/2014)**

- Changing the way Grackle works so that the primary chemistry data structure is no longer handled by the user but instead lives within the Grackle namespace.
- Porting all of the Grackle source to pure C from the original C++. This will make it significantly easier to directly call the Grackle functionality from within Fortran code. Created a proof of concept of Fortran calling C code in this manner and it works.

• **Proposal #2 (by Teyssier) --> This is put on a hold until Proposal #1 does fail. Gnedin thinks that none of these concerns would matter if Proposal #1 works.**

- Alternatively, one may consider falling back to recommending Grackle as a preferred and reliable option to compute cooling, but not as a strict requirement. We constrain the physics that goes into the code, but not the cooling routine itself. To ensure the compatibility between Grackle and non-Grackle cooling curves, we devise a one-zone test of cooling function and make sure that the results agree within an agreed-upon tolerance.

- [Step 1] If you cannot use Grackle, let us come up with a comparison test between Grackle and non-Grackle cooling (Romain can suggest one?).

- [Step 2] Make sure your cooling curve has the same physics as Grackle. Let us run it, and report the result. If the results are not sufficiently similar, who will investigate the issue? How much difference can we tolerate?

4. Useful Links

- (1) Gnedin's dummy Grackle API which only returns zeros (to test API by mimicking an adiabatic run): [link](#)
- (2) Grackle Google group: [link](#)
- (3) Grackle download & documentation: [link](#)